Click here to view the PDF version.

```julia
begin
    using Plots
    using PlutoUI
    using PlutoTeachingTools
    using LaTeXStrings
    using QuadGK
    using Printf
    using HypertextLiteral: @htl, @htl_str
end
```

Table of Contents

Numerical integration

⚠ TODO ⚠
Trapezoid formula
Simpson's formula

⚠ TODO ⚠
Error analysis

⚠ TODO ⚠
Optional: Extrapolation techniques
Optional: Node doubling
Optional: *A posteriori* error estimation

Numerical integration

Integration is an important operation in engineering and the physical sciences. However, frequently it is also difficult to integrate analytically. For such cases employing *numerical* integration techniques can be a good alternative, which sometimes turns out to be no less accurate.

Let's start with an easy analytic case. The integral $\int_0^1 e^x dx$ can be computed rather elegantly by analytical means, since the anti-derivative of e^x is again e^x. Therefore

$$\int_0^1 e^x dx = [e^x]_0^1 = e^1 - 1 \simeq 1.718281828459045.$$

```
exact = 1.718281828459045
```

```
exact = exp(1) - 1
```

To perform the integration numerically, we can employ the Julia package `QuadGK`, which implements an all-purpose numerical integration routine:

```
1.7182818284590453
```

```
let
    Q, error_estimate = quadgk(x -> exp(x), 0, 1)
    Q
end
```

As we observe, the difference to the analytical result is tiny and on the order of the floating-point precision.

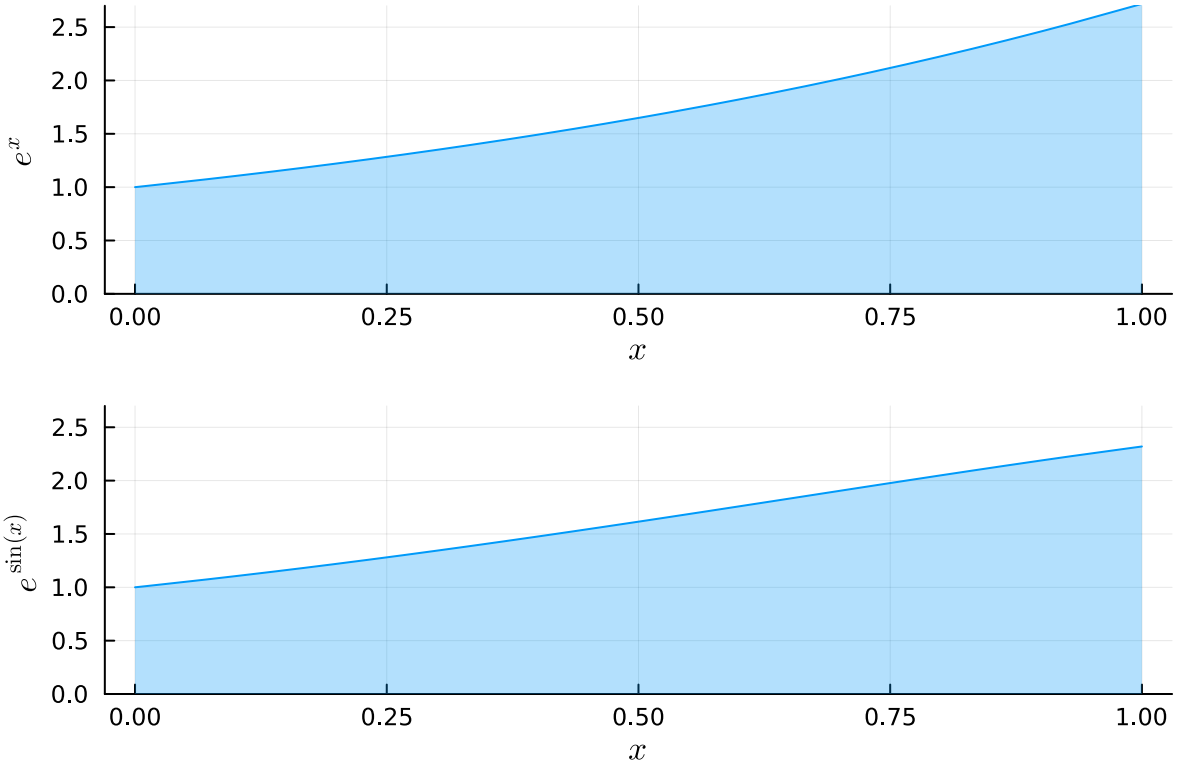
However, the numerical approach is much more flexible. For example $e^{\sin(x)}$ has no useful anti-derivative. Still the numerical approach works just as well to compute $\int_0^1 e^{\sin(x)} dx$:

```
1.6318696084180515
```

```
let
    Q, error_estimate = quadgk(x -> exp(sin(x)), 0, 1)
    Q
end
```

What is remarkable when looking at the graphs of these two functions is that they are very similar ... and for one case the area under the curve boils down to simple basic calculus and the

other is impenetrable analytically. However, from a numerical standpoint they are basically the same problem.




```
begin
    p = plot(exp, 0, 1, fill=0, fillalpha=0.3, xlabel=L"x", ylabel=L"e^x", ylims=(0, 2.7), label="")
    q = plot(x -> exp(sin(x)), 0, 1, fill=0, fillalpha=0.3, xlabel=L"x", ylabel=L"e^{\sin(x)}", ylims=(0, 2.7), label="")

    plot(p, q, layout=(2, 1))
end
```

The task of **numerical integration** is to approximate an integral $\int_a^b f(x)\,dx$. This is done by *sampling* the function at n carefully selected points t_i, $i = 0, \ldots n$ and linearly combining the outcome, that is $\int_a^b f(x)\,dx \approx \sum_{i=0}^n \alpha_i f(t_i)$. In general the quadrature nodes t_i can be distributed arbitrarily. However, for simplicity we will assume *equally spaced nodes* using the definitions

$$t_i = a + i\,h, \qquad h = \frac{b-a}{n} \qquad \text{where } i = 0, 1, \ldots, n \tag{1}$$

Given (1) we define:

Definition: Numerical integration formula

A **numerical integration formula** for the $n+1$ equispaced **quadrature nodes** t_i, $i = 0, \ldots, n$ is the set of **weights** $w_0, w_1, \ldots w_n$, such that for a certain class of functions $f : [a, b] \to \mathbb{R}$

$$\int_a^b f(x)dx \approx Q(f) = h \sum_{i=1}^{n} w_i f(t_i). \tag{2}$$

The weights are independent of f.

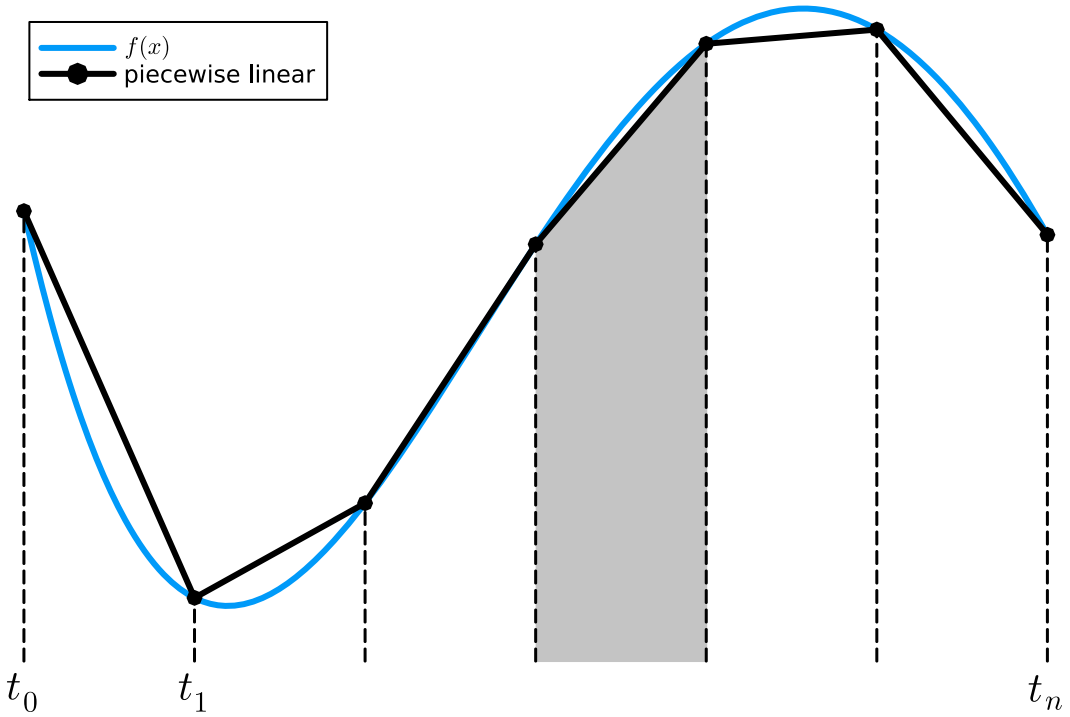
An older and still frequently used name for numerical integration is **quadrature**.

⚠ TODO ⚠

Motivate this as: We know how to find polynomial interpolations, we know how to integrate polynomials, so let's combine both to integrate arbitrary functions numerically

```
TODO("Motivate this as: We know how to find polynomial interpolations, we know how to integrate polynomials, so let's combine both to integrate arbitrary functions numerically")
```

Trapezoid formula



A simple idea to approximate the integral $I = \int_a^b f(x)dx$ is illustrated above. Using the $n+1$ equispaced nodes and respective data points $(t_i, f(t_i))$ we construct a piecewise linear polynomial interpolation $p_{1,h}$ (black line) to the integrand f. An approximation to I is then given by integrating the interpolant $p_{1,h}$:

$$\int_a^b f(x)\,dx \approx \int_a^b p_{1,h}(x)\,dx = \int_a^b \sum_{i=0}^{n} f(t_i) H_i(x)\,dx$$
$$= \sum_{i=0}^{n} f(t_i) \int_a^b H_i(x)\,dx.$$

The weights are therefore

$$w_i = \frac{1}{h}\int_a^b H_i(x)\,dx = \begin{cases} 1 & \text{if } i = 1, \ldots, n-1 \\ \frac{1}{2} & \text{if } i = 0 \text{ or } i = n \end{cases}$$

which can be easily checked using the area of triangles. Putting everything together we obtain

$$\int_a^b f(x)\,dx \approx T_n(f) := \frac{h}{2} f(t_0) + h f(t_1) + \cdots + h f(t_{n-1}) + \frac{h}{2} f(t_n)$$
$$= \frac{h}{2} f(t_0) + \sum_{i=1}^{n-1} h\, f(t_i) + \frac{h}{2} f(t_n)$$

A more geometric way of viewing the trapezoid formula is to think of it as summing the areas of the trapezoids defined by the quadrature nodes.

Definition: Trapezoid formula

The trapezoid formula is the numerical integration of the form (2) with weights

$$w_i = \begin{cases} 1 & \text{if } 0 < i < n \\ \frac{1}{2} & \text{if } i = 0 \text{ or } i = n \end{cases}$$

An implementation of the Trapezoid formula is:

```
trapezoid (generic function with 1 method)
```

```julia
function trapezoid(f, a, b, n)
    # f:  Function
    # [a, b]: Interval to integrate over
    # n:  Number of pieces to break the interval into
    h = (b - a) / n
    t = range(a, b, length=n+1)
    y = [f(tₙ) for tₙ in t]
    integral = h * (0.5y[1] + sum(y[2:n]) + 0.5y[n+1])
    (; integral, h)
end
```

Recall that in Theorem 4 of part 03 (Interpolation) we found that the piecewise polynomial interpolation shows quadratic convergence

$$\|f - p_{1,h}\|_\infty \leq Ch^2 \|f''\|_\infty.$$

With this in mind we can bound the error as

$$\begin{aligned} I - T_n(f) &= \int_a^b f(x)\,dx - \int_a^b p_{1,h}(x)\,dx \\ &= \int_a^b [f(x) - p_{1,h}(x)]\,dx \\ &\leq \|f - p_{1,h}\|_\infty \int_a^b dx \\ &\leq \underbrace{(b-a)\,C\,\|f''\|_\infty}_{=\tilde{C}} h^2 = \tilde{C}\,h^2 = O(h^2). \end{aligned}$$

A natural expectation is thus a **quadratic convergence**. We will see in the error analysis chapter that this is indeed the case.

But before we go into the theory let us first confirm this numerically. We consider the integral from earlier, i.e.

```
f (generic function with 1 method)
```

```
f(x) = exp(x)
```

and

$$I = \int_0^1 e^x\,dx = e - 1$$

which is approximately 1.718281828459045. Doubling the number of integration points 12 times we find:

```
ns = [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048]
```

```
ns = [2^i for i in 1:11]
```

The corresponding approximate integrals using the Trapezoid formula $T_n(f)$ and quadrature node distances are:

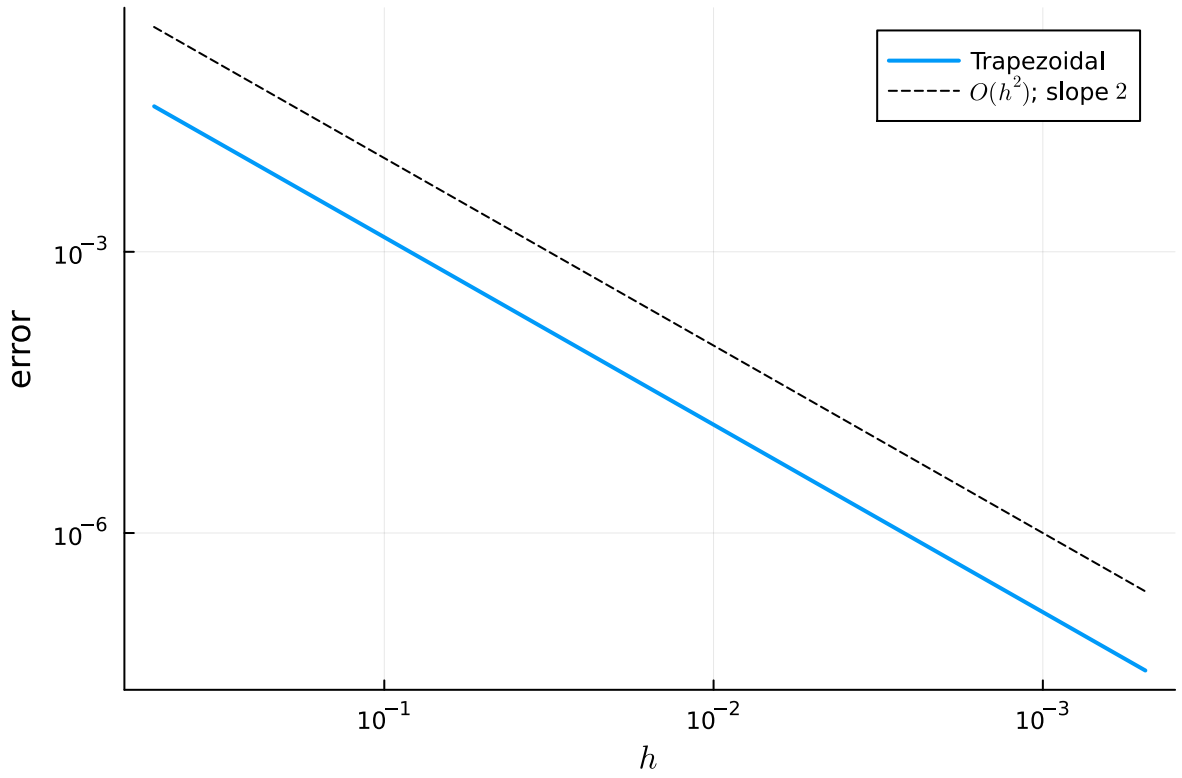
```julia
begin
    Tfs    = Float64[]
    hs     = Float64[]
    errors = Float64[]
    for n in ns
        res = trapezoid(f, 0, 1, n)
        Tf   = res.integral  # Trapezoidal approximation
        h     = res.h        # Quadrature node distances
        error = abs(Tf - exact)
        @printf "n =%4d   T(f) = %.10f   error = %.10f\n" n Tf error

        push!(Tfs, Tf)
        push!(hs, h)
        push!(errors, error)
    end
end
```

```
n =   2   T(f) = 1.7539310925   error = 0.0356492640
n =   4   T(f) = 1.7272219046   error = 0.0089400761
n =   8   T(f) = 1.7205185922   error = 0.0022367637
n =  16   T(f) = 1.7188411286   error = 0.0005593001
n =  32   T(f) = 1.7184216603   error = 0.0001398319
n =  64   T(f) = 1.7183167869   error = 0.0000349584
n = 128   T(f) = 1.7182905681   error = 0.0000087396
n = 256   T(f) = 1.7182840134   error = 0.0000021849
n = 512   T(f) = 1.7182823747   error = 0.0000005462
n =1024   T(f) = 1.7182819650   error = 0.0000001366
n =2048   T(f) = 1.7182818626   error = 0.0000000341
```

We notice that the error decreases roughly by a factor 4 when we double the number of quadrature nodes (i.e. half the distance h between the quadrature nodes). This again confirms the idea of a second-order convergence.

This becomes even clearer if we plot the error versus h in a log-log plot along with a line of slope 2:




```julia
let
    # Log-log plot where we flip the x-axis
    p = plot(hs, errors, yaxis=:log, xflip=true, xaxis=:log,
             xlabel=L"h", ylabel="error", label="Trapezoidal", lw=2)

    # Plot guiding line with slope 2 (quadratic convergence)
    plot!(p, hs, hs.^2, ls=:dash, c=:black, label=L"$O(h^2)$; slope $2$")
end
```

Simpson's formula

Simpson's formula takes the trapezoidal rule a little further and constructs a piecewise *quadratic* polynomial instead of a piecewise linear one. More precisely on every sub-interval $[t_i, t_{i+1}]$ $i = 0, \ldots, n$ we evaluate f on t_i, t_{i+1} and the midpoint $m_i = \frac{t_i + t_{i+1}}{2}$. Using $(t_i, f(t_i))$, $(m_i, f(m_i))$ and $(t_{i+1}, f(t_{i+1}))$ we interpolate a quadratic polynomial on $[t_i, t_{i+1}]$ leading to a picewise quadratic interpolant $p_{2,h}$ of the integrand on $[a, b]$. We approximate

$$\int_a^b f(x)\,dx \approx \int_a^b p_{2,h}(x)\,dx,$$

which is a little harder to compute and will be done as an exercise. The resulting formula is Simpson's formula

$$
\begin{aligned}
\int_a^b f(x)\,dx &\approx S_{2n}(f) \\
&= \sum_{i=1}^{n} \frac{h}{6}\left(f(t_{i-1}) + 4f(m_{i-1}) + f(t_i)\right) \\
&= \frac{h}{6} f(t_0) + \frac{h}{3}\sum_{i=1}^{n-1} f(t_i) + \frac{2h}{3}\sum_{i=0}^{n-1} f(m_i) + \frac{h}{6} f(t_n)
\end{aligned} \tag{3}
$$

and uses the equispaced nodes t_i for $i = 0, \ldots, n$ and m_i for $i = 0, \ldots n-1$, thus $2n+1$ nodes in total. Note that the nodal distance for this integration formula is $\tilde{h} = \frac{b-a}{2n} = \frac{h}{2}$. Simpson's formula thus also has the form of (2), even though the sequence of weights w_i is now more involved.

A Julia implementation is given below:

```
simpson (generic function with 1 method)
```

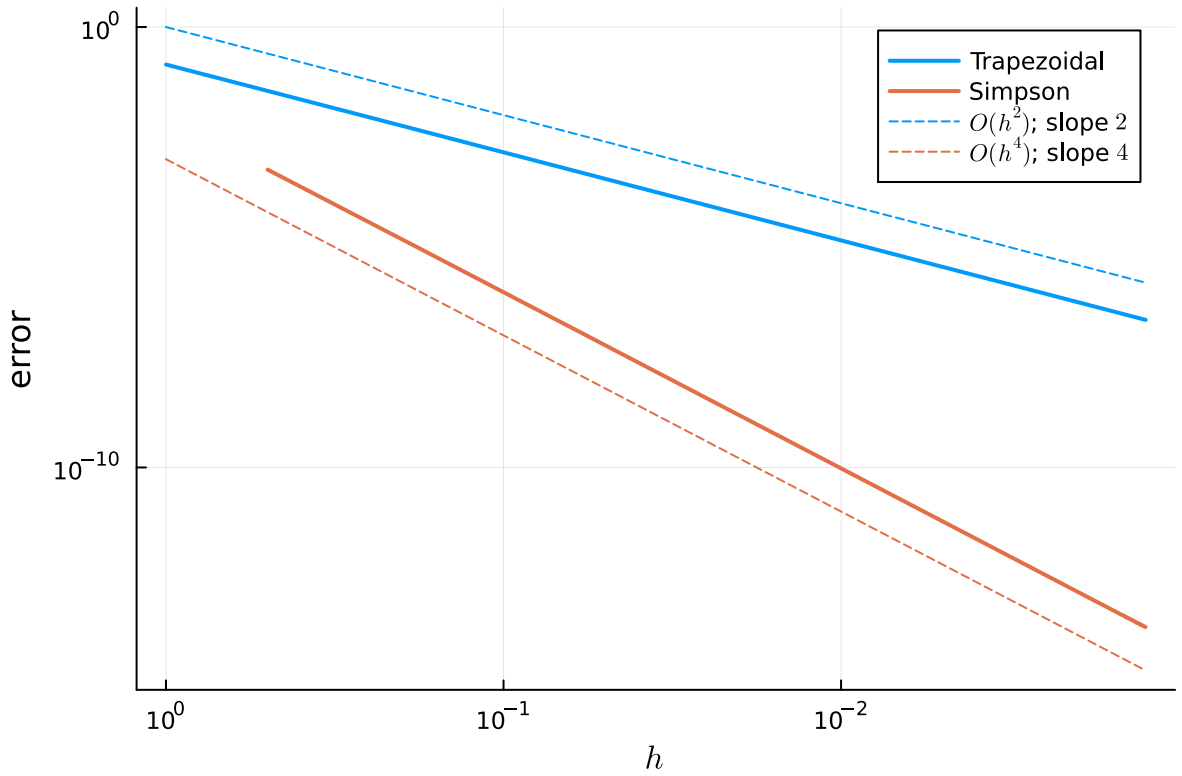
```julia
function simpson(f, a, b, n)
    # f:  Function
    # [a, b]: Interval to integrate over
    # n:  Number of pieces to break the interval into
    h = (b - a) / n
    t = range(a, b, length=n+1)        # Subinterval boundaries
    m = range(a+h/2, b-h/2, length=n)  # Subinterval midpoints

    # Evaluate f and compute approximation
    ft = f.(t)
    fm = f.(m)
    integral = h * (ft[1]/6 + sum(ft[2:n])/3 + 2sum(fm)/3 + ft[n+1]/6)

    (; integral, h=h/2)  # Note h/2 since the actual nodal distance is half of h
end
```

Comparing the error of the trapezoidal and Simpson's quadratures against the exact integral of $\int e^x dx$ we obtain a much faster convergence for Simpson's rule, numerically looking like an $O(h^4)$ convergence.

p_convergence =




```julia
p_convergence = let
    # Trapezoidal rule
    Tfs     = [trapezoid(f, 0, 1, n).integral for n in 1:3:800]
    hs_trap = [trapezoid(f, 0, 1, n).h        for n in 1:3:800]
    p = plot(hs_trap, abs.(Tfs .- exact);
             yaxis=:log, xflip=true, xaxis=:log,
             xlabel=L"h", ylabel="error", label="Trapezoidal", lw=2)

    # Simpson's rule
    Sfs      = [simpson(f, 0, 1, n).integral for n in 1:2:400]
    hs_simps = [simpson(f, 0, 1, n).h        for n in 1:2:400]
    plot!(p, hs_simps, abs.(Sfs .- exact), label="Simpson", lw=2)

    # Guiding lines
    plot!(p, hs_trap, hs_trap.^2, ls=:dash, c=1, label=L"$O(h^2)$; slope $2$")
    plot!(p, hs_trap, hs_trap.^4/1000, ls=:dash, c=2, label=L"$O(h^4)$; slope $4$")
end
```

⚠ TODO ⚠

Motivation of the degree of exactness by integrating exactly terms of the Taylor expansion with increasing power in x.

```
TODO(md"Motivation of the degree of exactness by integrating exactly terms of
the Taylor expansion with increasing power in $x$.")
```

Error analysis

To analyse the accuracy of a quadrature formula we start by introducing the concept of

Definition: Degree of exactness

A numerical integration formula $Q(f) = h \sum_{i=1}^{n} w_i f(t_i)$ (2) has a **degree of exactness** r if it integrates every polynomial with degree $\leq r$ exactly, i.e. if

$$Q(p) = \int_a^b p(x)\,dx \qquad \forall p \in \mathbb{P}_r,$$

but not for a $(r+1)$ -the degree polynomial.

Notice that we only need to check that the monomials x^s with $0 \leq s \leq r$ (s integer) are exactly integrated,

$$Q(x^s) = \int_a^b x^s\,dx \qquad \forall s \in \mathbb{N} \text{ with } 0 \leq s \leq r, \tag{4}$$

to ensure that r is the degree of exactness of Q. This is a consequence of the fact that the monomials $\{x^s\}_{s=0}^{r}$ form a basis for $\mathbb{P}_r$ and linaarity of the integral: let $p(x) = \sum_{s=0}^{r} a_s x^s \in \mathbb{P}_r$, then

$$
\begin{aligned}
Q(p) &= \sum_{i=1}^{n} w_i p(t_i) = \sum_{i=1}^{n} w_i \sum_{s=0}^{r} a_s x_i^s \\
&= \sum_{s=0}^{r} a_s \underbrace{\sum_{i=1}^{n} w_i x_i^s}_{\overset{(4)}{=} \int_a^b x^s \, dx} = \int_a^b \sum_{s=0}^{r} a_s x^s \, dx = \int_a^b p(x) \, dx,
\end{aligned}
$$

which means that every polynomial of degree $\leq r$ is exactly integrated, i.e. that Q has a degree of exactness r.

- The **Trapezoid formula** has been constructed by first using piecewise linear interpolation for the integrand f to give $p_{1,h}$. Then we *exactly* integrated the resulting interpolant $p_{1,h}$. If the original function f is already a linear polynomial, then $f = p_{1,h}$. We expect a **degree of exactness** $r = 1$, which can be shown easily to indeed be the case.
- Following this idea further we would expect Simpson's formula to have degree of exactness 2, which is interestingly not correct as one can show $r = 3$ — as we will do now.

Example: Simpson's formula has degree of exactness r=3.

We will confirm that Simpson's formula has degree of exactness $r = 3$.

Recall that the idea of Simpson's formula was to split the integral into n equally sized subintervals $[t_{i-1}, t_i]$ as

$$
\int_a^b f(x) \, dx = \sum_{i=1}^{n} \int_{t_{i-1}}^{t_i} f(x) \, dx
$$

and then on such a subinterval approximate

$$
\int_{t_{i-1}}^{t_i} f(x) \, dx \approx \frac{h}{6} \big(f(t_{i-1}) + 4 f(m_{i-1}) + f(t_i) \big). \tag{5}
$$

If the approximation in (5) is an equality on all subintervals for the monomilals with degree ≤ 3, then Simpson's formula is an equality for these monomials on the full interval $[a, b]$ as well. Inserting $f(x) = x^s$ into (5) we notice that our task is to show

$$
\int_{t_{i-1}}^{t_i} x^s \, dx = \frac{h}{6} \big(t_{i-1}^s + 4 m_{i-1}^s + t_i^s \big) \qquad \text{for } 0 \leq s \leq 3,
$$

but that this does not hold for $s = 4$.

To simplify our calculations we will assume that the interval $[t_{i-1}, t_i] = [-1, 1]$ with a length of $h = 2$, $t_{i-1} = -1$, $t_i = 1$, $m_{i-1} = 0$. This is a change one can make without loss of generality [1]. Inserting the values for h, t_{i-1}, t_i and m_i into the above expression we need to show that $0 \leq s \leq 3$

$$\int_{-1}^{1} x^s \, dx = \frac{2}{6}\left((-1)^s + 4 \cdot 0^s + (+1)^s\right) = \frac{(-1)^s + 4 \cdot 0^s + (+1)^s}{3},$$

but that this equality fails $s \geq 4$.

We have

$$\begin{aligned}
s = 0: &\quad \int_{-1}^{1} 1 \, dx = 2 = \frac{(-1)^0 + 4 \cdot 0^0 + 1^0}{3} && \checkmark \\
s = 1: &\quad \int_{-1}^{1} x \, dx = 0 = \frac{-1 + 0 + 1}{3} && \checkmark \\
s = 2: &\quad \int_{-1}^{1} x^2 \, dx = \frac{2}{3} = \frac{(-1)^2 + 4 \cdot 0^2 + 1^2}{3} && \checkmark \\
s = 3: &\quad \int_{-1}^{1} x^3 \, dx = 0 = \frac{(-1)^3 + 4 \cdot 0^3 + 1^3}{3} && \checkmark
\end{aligned}$$

but in contrast

$$s = 4: \quad \int_{-1}^{1} x^4 \, dx = \frac{2}{5} \neq \frac{(-1)^4 + 4 \cdot 0^4 + 1^4}{3} = \frac{2}{3}$$

Therefore Simpson's formula integrates exactly any third degree polynomial, but not the fourth degree polynomial, confirming that the degree of exactness is $r = 3$.

[1]:

> Even if $[t_{i-1}, t_i]$ is not $[-1, 1]$ then an appropriate linear transformation of the integration coordinate x can always be perfomed (e.g. by substitution of x in the integral), such that the integration does run over $[-1, 1]$. Therefore our result is also valid in the case of a general subinterval.

We summarise:

- **Trapezoid formula:** Degree of exactness $r = 1$
- **Simpson's formula:** Degree of exactness $r = 3$

⚠ TODO ⚠

The Simpson example needs some extra steps, e.g. discussing the substitution on the integral limits to get to $[-1, 1]$

```
TODO(md"The Simpson example needs some extra steps, e.g. discussing the substitution on the integral limits to get to $[-1, 1]$")
```

In line with the definition of algebraic convergence (and convergence order) for other approximation techniques we define the accuracy of a quadrature formula as:

Definition: Convergence order of numerical integration

A numerical integration formula $Q(f)$ of the form (2) with equally spaced quadrature notes of separation h is of **order** m if a constant C indepentent of h (but possibly dependent on f) exists, such that

$$\left| \int_a^b f(x)\,dx - Q(f) \right| \leq Ch^m$$

as long as the function f is sufficiently regular.

From our visual and numerical result we thus expect the Trapezoid formula to be a quadrature of order 2 and Simpson's formula of order 4. Comparing with the degree of exactness r this suggests a numerical integration formula to have degree $r+1$. Indeed one is able to prove:

Theorem 1

Given a numerical integration formula $Q(f)$ of form (2) with $n+1$ quadrature nodes of equal spacing $h = (b-a)/n$ which approximates $I = \int_a^b f(x)\,dx$. If $Q(f)$ has a degree of exactness r then it is of order $r+1$ as long as the function is of class C^{r+1} [2]. In particular we have

$$\left| \int_a^b f(x)\,dx - Q(f) \right| \leq C \, \|f^{(r+1)}\|_\infty \, h^{r+1}$$

where the constant $C = \frac{b-a}{2^r\,(r+1)!}$ depends neither on f nor on h.

[2]:

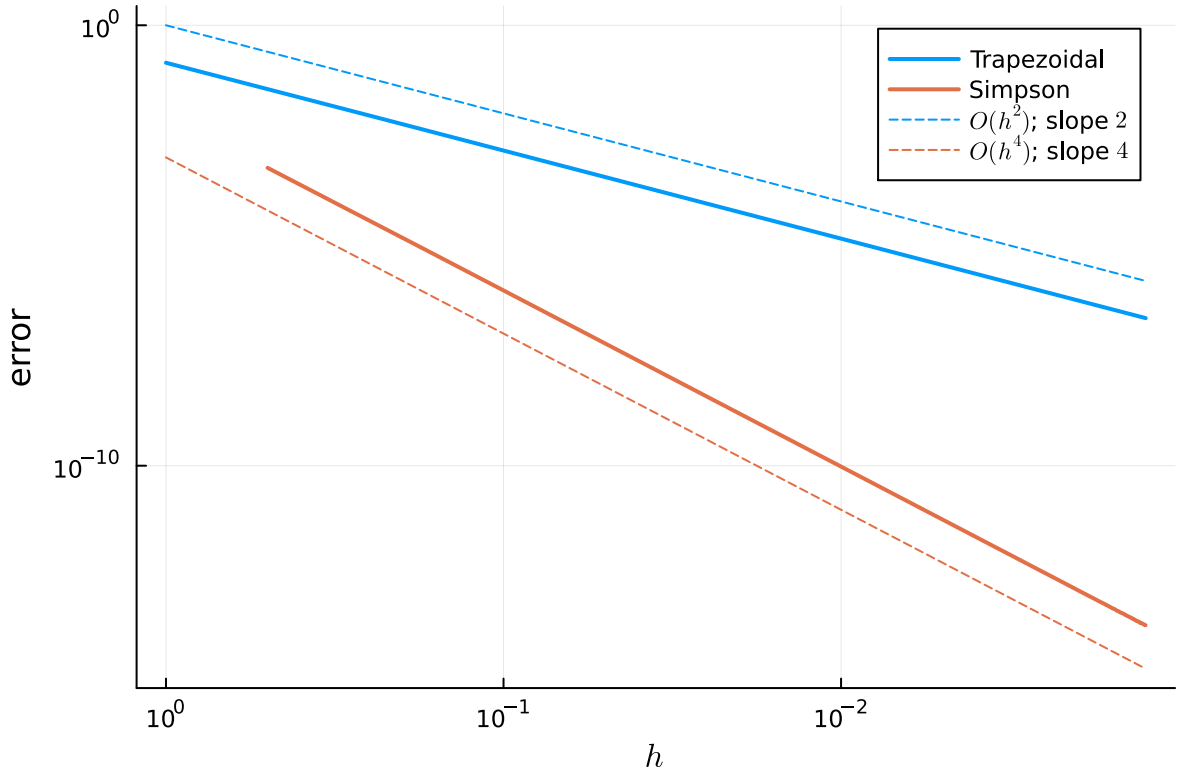
$r+1$ times continuously differentiable on the interval $[a,b]$

▶ **Show the proof**

Based on Theorem 1 we immediately find:

- **Trapezoid formula:** Degree of exactness $r=1 \Rightarrow$ order 2
- **Simpson's formula:** Degree of exactness $r=3 \Rightarrow$ order 4

This provides the theoretical confirmation of the behaviour we already observed visually in the above plot:



Optional: Extrapolation techniques

In numerical integration the computationally expensive step is usually the evaluation of the function f at the points of the quadrature nodes. Higher-order numerical integration formulas are therefore desirable to get most accuracy out of the function evaluations one performs. The natural question is thus how to construct higher-order quadrature formulas.

Here we will discuss a technique called **extrapolation** as one general method to obtain higher-order convergence. Suppose a quantity A_0 is approximated by an algorithm A_h with error expansion

$$A_0 = A_h + c_1 h + c_2 h^2 + c_3 h^3 + \cdots = A_h + c_1 h + c_2 h^2 + O(h^3). \tag{6}$$

Crucially it is not necessary to know the coefficients c_i, the only requirement is for them to be indepentent of h. If we now look at the approximation $A_{h/2}$ then

$$A_0 = A_{h/2} + \frac{c_1}{2} h + \frac{c_2}{4} h^2 + O(h^3) \tag{7}$$

Forming a linear combination $2 \cdot (7) - (6)$ we then obtain

$$A_0 = \underbrace{2A_{h/2} - A_h}_{=\tilde{A}_{h/2}} - \frac{c_2}{2} h^2 + O(h^3)$$

which defines an approximation algorithm $\tilde{A}_{h/2} = 2A_{h/2} - A_h$. Crucially while A_h is a first-order algorithm, $\tilde{A}_{h/2}$ is a *second-order* algorithm. This idea to cancel the lowest-order error term by linear combination of lower-order formulas of simple and half node spacing is usually referred to as **Richardson extrapolation**. Since the lowest-order error term cancels this scheme overall **increases the convergence order**.

Let us apply this to the Trapezoid formula for approximating the integral $I = \int_a^b f(x)\,dx$. We use $n+1$ quadrature nodes of equal separation $h = (b-a)/n$. Using the Euler–Maclaurin formula one can show for this case

$$I = T_n(f) + c_2 h^2 + c_4 h^4 + O(h^6), \tag{8}$$

where

$$c_2 = -\frac{1}{12}\big(f'(b) - f'(a)\big) \qquad c_4 = \frac{1}{740}\big(f'''(b) - f'''(a)\big).$$

Unless we are lucky enough to have $f'(b) = f'(a)$, we should thus expect a second-order convergence.

For convenience of notation we rewrite (8) in terms of the number of subintervals. Using $n = O(h^{-1})$ we obtain

$$I = T_n(f) + c_2\, n^{-2} + c_4\, n^{-4} + O(n^{-6}). \tag{9}$$

The Trapezoid formula with twice the number of subintervals (half the spacing) similarly reads

$$I = T_{2n}(f) + \frac{1}{4} c_2 \, n^{-2} + \frac{1}{16} c_4 \, n^{-4} + O(n^{-6}). \tag{10}$$

Using an appropriate linear combination we can again cancel out the second-order term. Specifically, define

$$S_{2n}(f) = \frac{1}{3}\Big(4\, T_{2n}(f) - T_n(f)\Big) \tag{11}$$

and consider $\frac{4}{3} \cdot (10) - \frac{1}{3}(9)$:

$$I = S_{2n}(f) - \frac{1}{4} c_4 \, n^{-4} + O(n^{-6}). \tag{12}$$

The quadrature formula $S_{2n}(f)$ is thus a 4-th order method. While not immediately clear (12) turns out to be identical to Simpson's formula (3), which we will check at the end of this section. In other words we can view **Simpson's formula** as the numerical integration formula obtained either

- by integrating a piecewise quadratic approximation of the integrand or
- when using Richardson extrapolation on the Trapezoidal rule.

We formulate a general procedure for Richardson extrapolation in the context of numerical integration:

Observation: General procedure for Richardson extrapolation

Given an error expansion of a quadrature Q_n with n subintervals

$$I = Q_n + c_p n^{-p} + O(n^{-q}), \tag{13}$$

where p is integer and $q > p$. Then a a **quadrature of at least order q** can be found by the linear combination

$$\widetilde{Q}_{2n} = \frac{2^p Q_{2n} - Q_n}{2^p - 1}. \tag{14}$$

This can be verified as follows: using twice the number of nodes in (13) we obtain

$$I = Q_{2n} + 2^{-p} c_p n^{-p} + O(n^{-q}). \tag{15}$$

Then considering the linear combination $\frac{2^p}{2^p-1}(15) - \frac{1}{2^p-1}(13)$ between the equations of single and double the number of subintervals we obtain

$$I = \widetilde{Q}_{2n} + O(n^{-q}),$$

i.e. that $\widetilde{Q}_{2n}$ is of order at least q.

Note that Simpson's formula (11) turns out to be another error expansion of the form (6), so we can extrapolate another time using Simpson's rule with $2n$ and $4n$ quadrature subintervals. From (14) we note the appropriate linear combination to be

$$R_{4n}(f) = \frac{1}{15}\Big(16 S_{4n}(f) - S_{2n}(f)\Big), \tag{16}$$

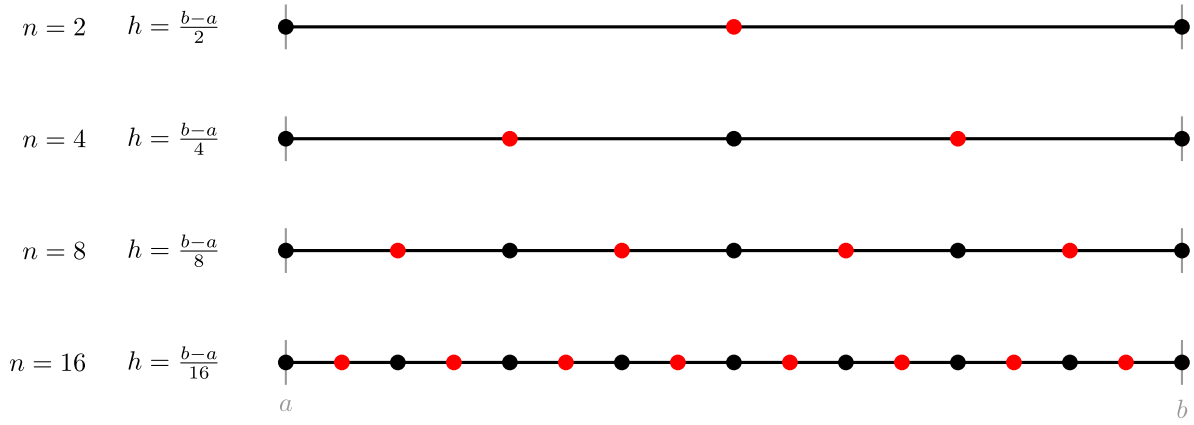
which is a 6-th order numerical integration formula. This process can of course be repeated using $8n$, $16n$, $\ldots$ subintervals, leading to higher and higher quadrature orders. One usually calls this **Romberg integration**, which we will not present in full generality here.

▶ **Details on the equality of the two forms of Simpson's formula**

Optional: Node doubling

If we consider the integration formula R_{4n} we notice that R_{4n} depends on S_{4n} and S_{2n}, which in turn depend on T_n, T_{2n} and T_{4n}. Continuing along the Romberg integration procedure we would recursively perform more and more levels of extrapolation. At the next level we also require T_{8n}, then T_{16n} and so on, such that in each step the number of nodes to consider roughly doubles.

In principle one could also construct other recursive extrapolation procedures and consider the linear combination using other quadrature node counts. However, there is an added benefit from specifically using this node doubling scheme. As shown in the figure below dividing the node spacing by half introduces new nodes only at the midpoints (shown in red). Notably on most quadrature nodes the function values are thus already known from the previous level and can be re-used during the extrapolation procedure. Overall moving from h to $h/2$ (respectively from $n+1$ to $2n+1$ nodes) only about half of the nodes are new and need to be evaluated.



To make this explicit recall (17). In this expression we reformulated the Trapezoid rule with $2n$ subintervals in terms of quantities of the Trapezoid rule with n subintervals, that is the quadrature nodes $t_i = a + i\,h$ (for $i = 0, \ldots, n$), the nodal spacing $h = (b-a)/n$ and the midpoints $m_i = \frac{t_i + t_{i+1}}{2}$:

$$T_{2n}(f) = \frac{h}{4} f(t_0) + \frac{h}{2} \sum_{i=0}^{n-1} f(m_i) + \frac{h}{2} \sum_{i=1}^{n-1} f(t_i) + \frac{h}{4} f(t_n).$$

Noting $m_i = a + (2i+1)\,\frac{h}{2}$ for $i = 0, \ldots, n-1$ and using the definition of $T_n(f)$ this can be reformulated as

$$T_{2n}(f) = \frac{1}{2} T_n(f) + \frac{h}{2} \sum_{i=0}^{n-1} f\left(a + (2i+1)\,\frac{h}{2}\right). \tag{18}$$

Since $a + (2i+1)\,\frac{h}{2} = a + (2i+1)\,\frac{b-a}{2n}$ are just the *odd* quadrature nodes of the $T_{2n}(f)$ quadrature formula this makes explicit that only these n new nodes need to be evaluated when going from $T_n(f)$ to $T_{2n}(f)$.

Let's see this in an example. We want to compute

$$V = \int_0^2 x^2 e^{-2x}\, dx$$

using extrapolation. First we use `quadgk` to get an accurate value:

```
begin
    g(x) = x^2 * exp(-2x)
    a = 0
    b = 2
    V, _ = quadgk(g, a, b, atol=1e-14, rtol=1e-14)
end;
```

```
0.1904741736116139
```

```
V
```

Based on the Trapezoid formula on $n = 20$ subintervals we get a first estimate:

```
T20 = 0.19041144993926787
```

```
T20 = let
    n = 20
    h = (b - a) / n
    t = h * (0:n)
    y = g.(t)
    T20 = h*y[1]/2 + h*sum(y[2:n]) + h*y[n+1]/2
end
```

Now we double to $n = 40$, but we only need to evaluate the odd nodes:

```
T40 = 0.19045880585951175
```

```
T40 = let
    n = 40
    h = (b - a) / n
    t = h * (0:n)
    y_odd = g.(t[2:2:n])
    T40 = T20/2 + h*sum(y_odd)
end
```

We repeat a second time and double n again:

```
T80 = 0.1904703513046443
```

```
T80 = let
    n = 80
    h = (b - a) / n
    t = h * (0:n)
    y_odd = g.(t[2:2:n])
    T80 = T40/2 + h*sum(y_odd)
end
```

Using (11) we can perform a first level of extrapolation and get the two Simpson values $S_{40}(f)$ and $S_{80}(f)$:

```
S40 = 0.19047459116625973
```

```
S40 = (4T40 - T20) / 3
```

```
S80 = 0.19047419978635513
```

```
S80 = (4T80 - T40) / 3
```

Finally, we perform one more level of extrapolation to get the sxth-order accurate result $R_{80}(f)$:

```
R80 = 0.1904741736943615
```

```
R80 = (16S80 - S40) / 15
```

We compute all errors to 6 digits:

```
begin
    eT = round.(V .- [T20 T40 T80]; digits=10)
    eS = round.(V .- [S40 S80];     digits=10)
    eR = round.(V .- [R80];         digits=10)
end;
```

and summarise them in a table along with the number of function evaluations:

number of evals	order 2	order 4	order 6
21	6.27237e-5		
41	1.53678e-5	-4.176e-7	
81	3.8223e-6	-2.62e-8	-1.0e-10

We notice that the 6th order **result obtained using Richardson extrapolation** is about **twice as accurate** even though it uses the **same number of function evaluations**.

Since the cost of function evaluation is usually dominating in numerical integration, the take-away is that one should **never just use low-order quadratures**, but always **employ extrapolation techniques.**

Optional: *A posteriori* error estimation

If we want to numerically evaluate an integral $I = \int_a^b f(x)\,dx$ using a chosen quadrature formula Q_n, a natural question to ask is how many quadrature nodes n are necessary to obtain an approximation of the the integral I within a predefined error tolerance ϵ.

In the previous sections we discussed, that Richardson extrapolation provides a receipe to obtain a higher-order quadrature $\widetilde{Q}_{2n}(f)$ from the numerical integration values $Q_n(f)$ and $Q_{2n}(f)$. As a result $\widetilde{Q}_{2n}(f)$ is in general more accurate than $Q_{2n}(f)$, which motivates to employ the difference

$$\eta_{2n} = |\widetilde{Q}_{2n}(f) - Q_{2n}(f)| \tag{19}$$

as an estimate of the error committed by the formula $Q_{2n}(f)$. Indeed assuming an error expansion (13), i.e. $I = Q_n(f) + c_p n^{-p} + O(n^{-q})$, and and a corresponding Richardson extrapolation (14) one verifies

$$\begin{aligned} |I - Q_{2n}(f)| \leq |I - \widetilde{Q}_{2n}(f)| + |\widetilde{Q}_{2n}(f) - Q_{2n}(f)| &= \underbrace{\eta_{2n}}_{=O(n^{-p})} + \underbrace{|I - \widetilde{Q}_{2n}(f)|}_{=O(n^{-q})} \\ &= \eta_{2n} + O(n^{-q}) \end{aligned}$$

Therefore a simple adaptive strategy is to start with $2n$ nodes by computing and $Q_n(f)$ and $Q_{2n}(f)$. Then check whether $\eta_{2n} \leq \epsilon$ is satisfied and if this is not the case keep doubling the number of nodes until it is. Sticking to a single layer of extrapolation for simplicity we obtain the following algorithm:

Algorithm: Adaptive quadrature based on Richardson extrapolation

Given the problem to compute $\int_a^b f(x)\,dx$, quadrature formula $Q_n(f)$ an initial number of subintervals n_0 and a requested tolerance ϵ, compute:

- Initialise $n = n_0$, $h = (b-a)/n_0$
- Initial estimate of integral $I_n = Q_n(f)$
- While $\eta_n > \epsilon$ iterate:
  - Update $h = \frac{b-a}{2n}$
  - $I_{2n} = Q_{2n}(f)$ (using updated node distance h)
  - Extrapolate $\tilde{I}_{2n} = \frac{2^p I_{2n} - I_n}{2^p - 1}$ using (14)
  - $\eta_{2n} = |\tilde{I}_{2n} - I_{2n}|$
  - Update $n = 2n$

Applied to the Trapezoid formula this is implemented as follows:

```
trapezoid_adaptive (generic function with 1 method)
```

```julia
function trapezoid_adaptive(f, a, b; n₀=2, tol=1e-12)
    # f:  Function
    # [a, b]: Interval to integrate over
    # n₀: Initial number of subintervals (i.e. n₀+1 quadrature nodes)
    # tol: Desired tolerance
    n = n₀
    h = (b - a) / n₀  # Node separation
    t = (0:n) .* h   # Quadrature nodes
    y = f.(t)        # Evaluate function on all nodes
    Tₙ = h * (0.5y[1] + sum(y[2:n]) + 0.5y[n+1])  # Trapezoid formula with n₀ nodes

    extrapolated = 0.0
    ηₙ = 10tol
    while ηₙ > tol
        h = (b - a) / 2n  # New node separation
        t = (0:2n) .* h   # New set of quadrature nodes
        y_odd = f.(t[2:2:2n])  # Evaluate only at odd nodes

        # Use node doubling formula (18) to evaluate Trapezoid formula with 2n nodes.
        T₂ₙ = Tₙ/2 + h * sum(y_odd)

        # Perform extrapolation: Note that p = 2 for the Trapezoid formula
        extrapolated = (4T₂ₙ - Tₙ) / 3

        n  = 2n
        Tₙ = T₂ₙ
        ηₙ = abs(extrapolated - T₂ₙ)
    end

    (; integral=extrapolated, h, n)
end
```

We test this works as expected using our example from the node doubling section:

```
▸ (integral = 0.190474, h = 0.03125, n = 64)
```

```julia
(; integral, h, n) = trapezoid_adaptive(g, 0, 2; tol=1e-5)
```

The error is indeed below the requested tolerance:

```
6.386761788879092e-8
```

```julia
abs(integral - V)
```

The next step to improve this approach would be to additionally employ Romberg integration, i.e. to recursive extrapolate not only from $T_{2n}(f)$ & $T_{4n}(f)$ to $S_{4n}(f)$, but to also employ $T_n(f)$

and evaluate the 6-th order formula $R_{4n}(f)$ like we did in the numerical example of the node doubling section. However, these ideas are out of scope for us.

Numerical analysis

1. Introduction
2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Interpolation
6. Direct methods for linear systems
7. Iterative methods for linear systems
8. Eigenvalue problems
9. Numerical integration
10. Numerical differentiation
11. Initial value problems
12. Boundary value problems